

JavaScript Stacks & Queues

A COMPLETE GUIDE

The Most Complete Full Stack
Developer Roadmap

www.thedevspace.io

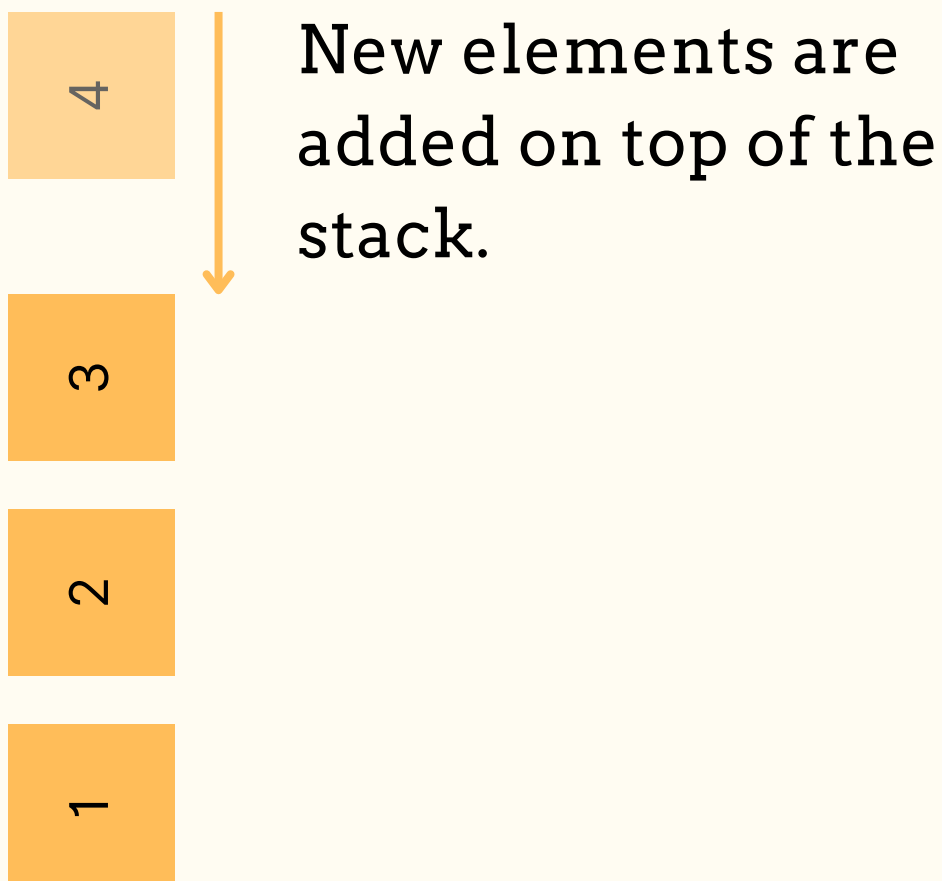
OVERVIEW

``push``, ``pop``, ``shift``, and ``unshift`` are four common array mutators in JavaScript.

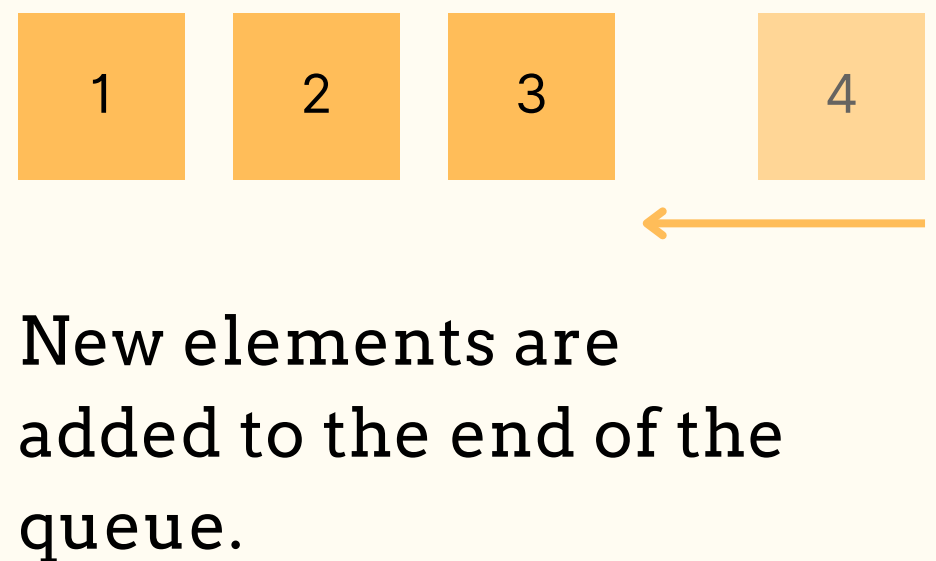
These methods add or remove elements from the ends of an array.

All four methods are all mutation methods, meaning they change the original array.

Stack

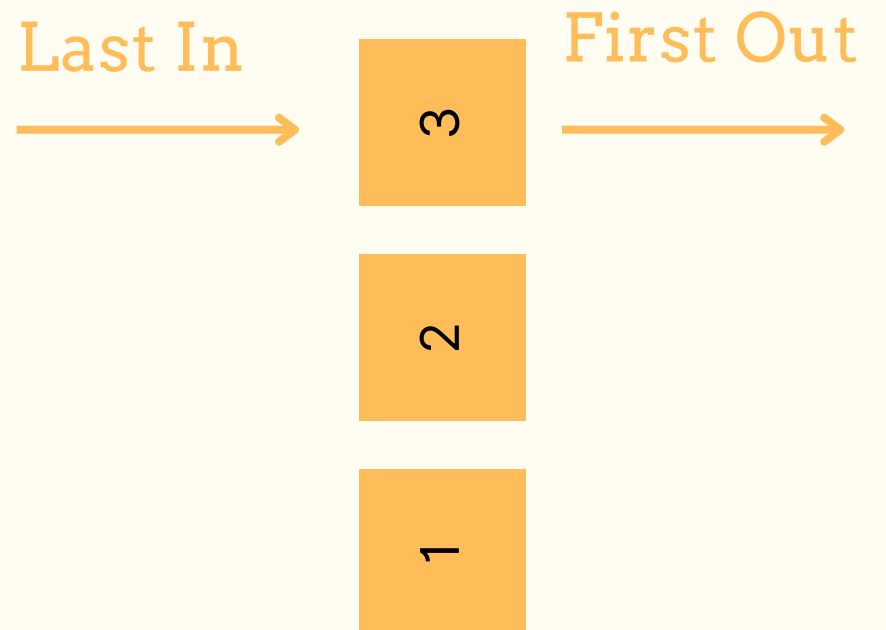


Queue

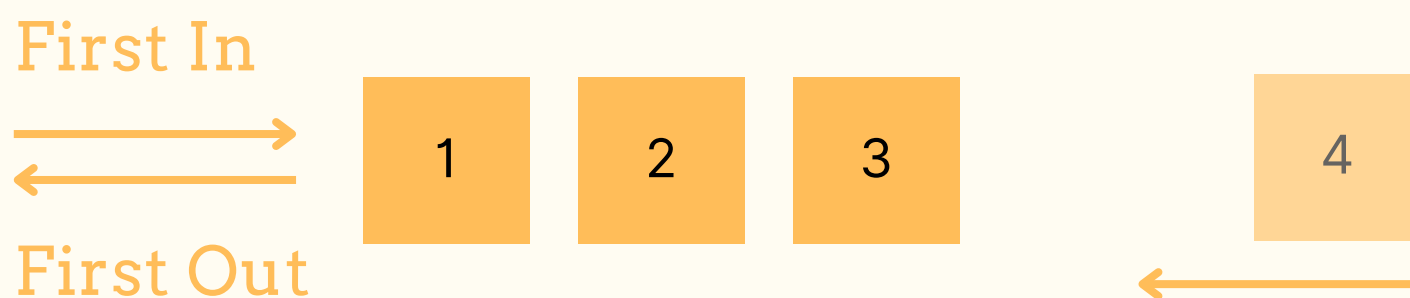


- `push()`: Adds elements to end
- `pop()`: Removes last element
- `shift()`: Removes first element
- `unshift()`: Adds elements to beginning

Use `push`/`pop` for stack operations at the end of the array (fast).



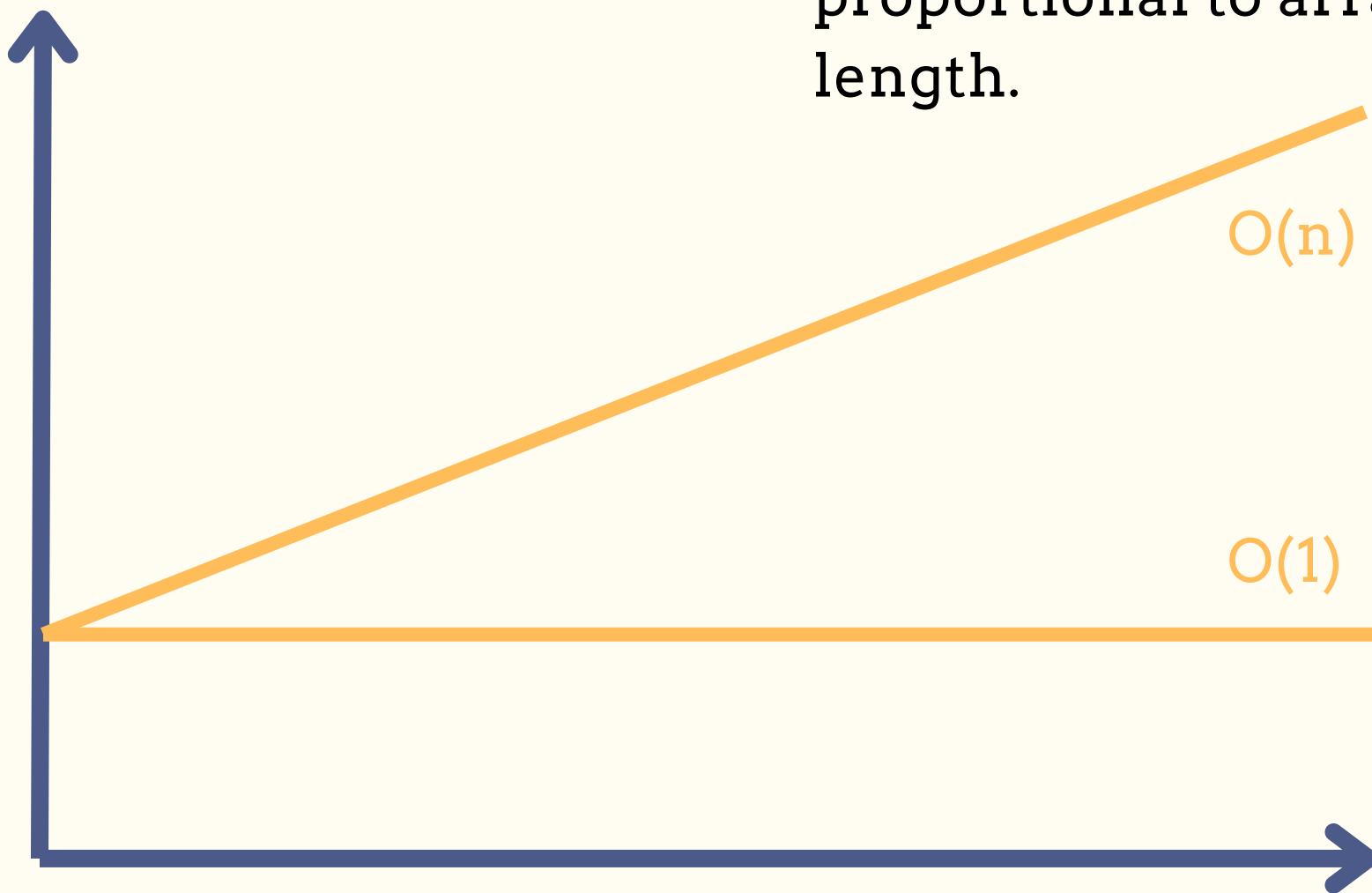
Use `shift`/`unshift` for queue operations at the start of the array (slow for large arrays).



COMPLEXITY & PERFORMANCE

``push`` and ``pop`` are generally $O(1)$, as they operate at array end.

``shift`` and ``unshift`` are $O(n)$. They reindex existing elements, and the cost is proportional to array length.



For high-performance queues, prefer a circular buffer or keep an index instead of frequently using ``shift``/``unshift``.

RETURN VALUES

`push(...items)` returns the new length of the array.

```
1 const a = [1, 2, 3];
2
3 // push: append one or more items at the end
4 a.push(4); // -> 4 (new length)
5 // a is now [1, 2, 3, 4]
```

```
1 // pop: remove and return last item
2 const last = a.pop(); // -> 4
3 // a is now [1, 2, 3]
```

`pop()` returns the removed element (or `undefined` if empty).

`unshift(...items)` returns the new length of the array.

```
1 // unshift: add items at start
2 a.unshift(0); // -> 4 (new length)
3 // a is now [0, 1, 2, 3]
```

```
1 // shift: remove first item
2 const first = a.shift(); // -> 0
3 // a is now [1, 2, 3]
```

`shift()` returns the removed element (or `undefined` if empty).

STACKS & QUEUES

Use `push` + `pop` to implement a stack (LIFO):



```
1 const stack = [];  
2 stack.push(1);  
3 stack.push(2);  
4 const v = stack.pop(); // pop -> 2
```

Use `push` + `shift` to implement a queue (FIFO):



```
1 const queue = [];  
2 queue.push(1);  
3 queue.push(2);  
4 const v = queue.shift(); // shift -> 1
```

Implement index-based queue for heavy workloads:

```
1 // inefficient (reindexes on every shift)
2 const q1 = [1,2,3];
3 while (q1.length) {
4   doWork(q1.shift()); // O(n) per shift -> O(n^2) overall for many shifts
5 }
6
7 // better: use head index
8 const q2 = [1,2,3];
9 let head = 0;
10 while (head < q2.length) {
11   doWork(q2[head++]); // O(1) per step
12 }
```

